例題で理解するPython

川又 政征（東北職業能力開発大学校），
阿部 正英（石巻専修大学），八巻 俊輔（東北大学）

本資料は Pythonの一つのプログラムを読むことでディジタル信号処理のために必要な Python の概要を短時間で直感的に理解することを目的としている．Pythonを勉強したいがマニュアルを読む時間がない人あるいはPythonの概要をすぐに知りたい人のための資料である．

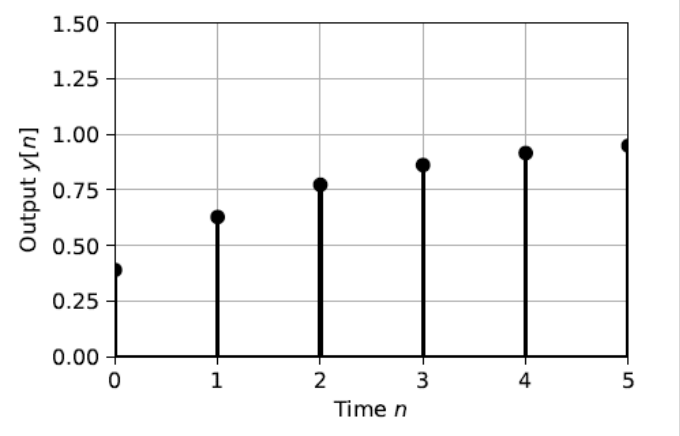
1. Pythonのプログラムの例

以下のプログラムを考えよう．これは簡単な1次ディジタルフィルタの単位ステップ応答を繰り返し計算により求め，その出力を印字し，グラフとして描画するプログラムである．

```
# ディジタルフィルタの単位ステップ応答 (Python対応DSP，プログラム 1.2)
import numpy as np                  # 数値計算ライブラリを 別名 np として読み込み
import matplotlib.pyplot as plt     # グラフ描画ライブラリを別名 plt として読み込み

beta = 0.39                                 # パラメータの設定
alpha = 0.61                                # パラメータの設定
nend = 5                                    # 終了時刻

n = np.arange(0, nend + 1)                  # 時刻の範囲
x = np.ones(nend + 1)                       # 単位ステップ入力
y = np.zeros(nend + 1)                      # 出力信号の初期化
y[0] = beta * x[0]                          # 出力y[0]の計算

for m in range(1, nend + 1):                # 時刻 m = 1〜nendまで繰り返し
    y[m] = alpha * y[m - 1] + beta * x[m]   # 出力 y[m] の計算(差分方程式)

print('y = ¥n', y)                          # 出力信号値の表示

fig = plt.figure()                          # Figure オブジェクトの生成
ax = fig.add_subplot(1, 1, 1)               # 1 行 1 列に分割し，1 番目に図示
ax.stem(n, y)                               # 出力の図示
ax.set_xlim(0, nend)                        # x 座標軸の範囲の設定
ax.set_ylim(0, 1.5)                         # y 座標軸の範囲の設定
ax.grid()                                   # 格子の設定
ax.set_xlabel('Time $n$')                   # x 座標軸名の設定
ax.set_ylabel('Output $y[n]$')              # y 座標軸名の設定
plt.show()                                  # グラフの実際の表示
```

このプログラムを実行すると，以下のように出力 y の数値が印字され，グラフが描画される．

```
y =
 [0.39      0.6279     0.773019   0.86154159 0.91554037 0.94847963]
```



2. プログラムの解読

以下ではこのプログラムを読み，Pythonの概要を短時間で直感的に理解することにする．

```
1:  # ディジタルフィルタの単位ステップ応答
```

はコメントの開始のための記号である．# から行末までの文字がコメントとなり，実行はされない．コメントは行のどの位置からでも始められる．

```
2:  import numpy as np
3:  import matplotlib.pyplot as plt
```

Pytyhonには数値計算やグラフ描画のための様々な関数モジュールが提供されている．数値計算ライブラリnumpyとグラフ描画ライブラリmatplotlib はたびたび用いられる関数モジュールである．import はライブラリを読み込み，別名を割り当てる命令である．2: では数値計算のライブラリ numpyを読み込み，別名 np を付けている．ライブラリnumpy 中の関数 foo は numpy.foo として呼び出すことができるが，このままでは関数名が長くなるので慣習的に別名 np を使って np.foo として関数を呼び出す．3: のグラフ描画ライブラリ matplotlib.pyplot についても同様である．

```
5:  beta = 0.39
6:  alpha = 0.61
7:  nend = 5
```

右辺の定数値を左辺の変数に代入している．Pythonでは，変数名とその型を宣言することなく整数，実数，複素数，ベクトル，行列をすぐに変数に代入できる．

```
9:  n = np.arange(0, nend + 1)
```

np.arange(0, nend+1) は, 0からnend（この場合には nend = 5）までの整数値のベクトル[0, 1, 2, 3, 4, 5]を生成し，これが変数 n に代入される．np.arange(0, nend + 1) から生成されるベクトルにおいて注意すべきことは, nendが終了値であることである．またベクトル n の各要素は n[0], n[1], …, n[5] のように変数名とカギ括弧[ ], 整数インデックスで指定される．ベクトル（あるいは行列）のインデックスは 0 から始まることに十分注意すること．

```
10:  x = np.ones(nend + 1)
11:  y = np.zeros(nend + 1)
```

np.ones(nend+1) は要素がすべて 1. である 1xnend ベクトル [1., 1., 1., 1., 1., 1.] を生成し，これが変数 x に代入される．ここでベクトル x は単位ステップ入力として用いられている．np.zeos(nend + 1) は，要素がすべて 0. である 1xnend ベクトル [0., 0., 0., 0., 0., 0.]を生成し，これが y に代入される．ここでベクトル y はディジタルフィルタの出力として初期化されている．

```
12:  y[0] = beta * x[0]
```

実数 beta をベクトル x の最初の要素 x[0]に乗じて，その結果をベクトル y の最初の要素 y[0] に代入する．

```
14:  for m in range(1, nend + 1):
15:      y[m] = alpha * y[m - 1] + beta * x[m]
```

for 文は繰り返しの制御構文である．14: において m が 1 から nend までの範囲で一つずつ増加するたびに 15：の行が繰り返し実行される．14-15: において 1 次の差分方程式の計算が行われている．

for 文の繰り返しの対象は，for の次の行からのコードブロックである．コードブロックは，空白 4 ヶでインデントされた一つ以上のコードの集まりである（ただしこの例ではコードブロックは 15: の一行である）．ここで重要なことは空白 4 ヶのインデントはコードブロックが繰り返しの対象となるための文法上必須のインデントであることである．この必要性により，Pythonでは for 文において（while 文や if 文においても）強制的にインデントが行われることになり，プログラムの可読性が高まる．

```
17:  print('y = ¥n', y)
```

print 文はディスプレイに文字や数値を印字する命令である．シングルクーテーション‘‘で囲まれた文字は文字列を表す．17: ではディスプレイに文字列 ‘y = ¥n’ とベクトル y を出力する．ただし ¥n は表示されず，改行コードとして作用する．

```
19:  fig = plt.figure()
```

19:-26: では横軸を n として，縦軸を y とするグラフの描画のための準備が行われている．まず，19: の plt.figure() により描画領域を確保し，それが fig に代入されている（fig と名付けられている）．括弧()の中で様々なパラメータを指定することができるが，ここには何も

パラメータを入れていないので，デフォルトの描画領域が確保される．この段階では例えばA4 判の白紙（名称 fig ）が確保されたと思えばよい．

```
20: ax = fig.add_subplot(1, 1, 1)
```

描画領域 fig にグラフ用紙を追加する．ここで(1, 1, 1) は描画領域を 1 行 1 列に分割し，その 1 番目（すなわち，この場合には描画領域全体を使い）の範囲をaxに代入している（axと名付けている）．これにより，描画領域 fig の中にグラフ用紙 ax が準備されたと考えればよい．

```
21: ax.stem(n, y)
```

グラフ用紙 ax に横軸変数n, 縦軸変数 yとして，茎状(stem)のグラフを描く．ベクトル n と y の長さ（要素数）は等しくなければならない．

```
22: ax.set_xlim(0, nend)
23: ax.set_ylim(0, 1.5)
```

グラフ用紙 ax の x 軸（横軸）の範囲を 0〜nend に制限する．同様に y 軸（縦軸）の範囲を 0〜1.5 に制限する．

```
24: ax.grid()
```

グラフ用紙 ax に縦横の格子を入れる．格子の色や間隔は括弧()の中で指定するが，ここでは何も指定がないのでデフォルトの指定に任せることを意味する．

```
25: ax.set_xlabel('Time $n$')
26: ax.set_ylabel('Output $y[n]$')
```

グラフ用紙 ax の x 軸ラベルを ’Time n’ とし，y 軸ラベルを ‘Output $y[n]$’ とする．ここで，シングルクォーテーション ‘ ‘ で囲まれた文字は文字列を表す．二つの $ $ で囲まれた文字は数式モードの文字（この場合はイタリック文字）を意味する．数式モードではフォント，上付き，下付き，特殊記号などが指定できる．

```
27: plt.show()
```

19-26: のコードはグラフの表示の準備をしているだけであるため，この段階ではグラフは表示されない．27: の plt.show() によってグラフが実際に表示される．

注意： Python の統合開発環境 Spyder では plt.show() を実行しなくても図が表示されるようになっているため，Python 対応 ディジタル信号処理」のプログラムでは plt.show() を省略している．

終